1. **Algorithm Descriptions: Describe each of the 5 implemented algorithms. For the custom algorithms, explain how classic sorting algorithms learned in class were either combined or modified. Include the theoretical best-, worst-, and average-case time complexities, as well as space complexity analysis for all 5 algorithms.**

**Merge Sort:**

On paper, merge sort is supposed to split the array elements until they're by themselves, then uses recursion to merge them back in a sorted error.

While merge sort is incredibly fast, with all best, worst, and average cases having O(n * log(n)) time, it is spatially more expensive; as a temp array has to be made to store the sorted elements, costing us a space complexity of O(n). Where n is the number of elements being sorted.

This is shown in my code when I created a movie array variable named temp with equal size to the movie input. Since merge uses recursion, our base case would be if the left side was greater than or equal to the right side, then our splitting would be done.

The next step was to create temporary variables to track the left, right, and more importantly, the midpoint. These pointers are later used for splitting halves.

Then using "mergeSortHelper", it recursively split them until elements are by themselves.

Then to finally to be sorted in one single array, the use of a while loop and if conditions tells us that while the left side is less than midpoint, and the right pointer is less than right, if the left side is smaller, use the starter pointer to copy that element into the temp array, else, copy the right side.

The while loop condition set earlier stops when either condition is met, thus there is potential for missing a movie if the sides are equal. So using an extra 2 sets of while loops, it ensures that neither side are missing elements. Then the last step is to copy the temp array into the movies array. Since this algorithm is pretty well known, I don't think I made any personal modifications and it is pretty standard.

**Quick Sort:**

The quick sort algorithm has the best and average case of O(n * log(n)) and a worse case of O(n^2). The time complexity for this algorithm is O(n * log(n)) for best and average case, and O(n) for the worst case.

Quick sort is by far the newest algorithm I have ever encountered and picking the pivot was the most personal change/modification I made to this algorithm, where I always picked the last element to be the pivot. In class, this was how it was taught to us from what I remember, with every pseudo-code example, the last element was the pivot, however, I am aware that that is not always the case.

The implementation of this algorithm is that it chooses the last element as the pivot, and recursively calls itself for the left and right side. Given this is recursion, we need a base case to break out, that condition being if the right index is less than the left index. Using a for loop, it partitions the elements such that the left half is less than the pivot, and the right half is

on the right of the pivot. It's not sorted yet, but this is almost like the splitting stage for merge. Then, using an index pointer, it gets the pivot into its correct position, with respect to the left and right side. And finally, quickSortHelper recursively calls the left/right sides to sort the partitions.

**Sequence Stitcher**

This algorithm is a combination of merge sort and bubble sort, with the best, worst and average time complexities being O(n log(n)); and a space complexity of O(n). How this algorithm essentially works is that if the size of the array is greater than 16, it uses bubble sort; and for arrays greater than 16, it uses merge sort. The bubble sort algorithm does include an early break, improving time complexity if no elements swap through the first/final pass. Just like a traditional bubble sort, it uses nested loops to compare adjacent elements and swaps respectively in a sorted order. And for the merge sort portion, if an array is greater than 16, it uses the same logic as the merge sort from earlier, where we recursively split with respect to the min, max, and midpoint values, copies the elements in a sorted order into a temp array, then copy the temp array into the movies array, thus the time complexity is O(n).

**Adaptive Reel Sort**

This algorithm is a combination of quick and insertion sort. The partition function defined below is what makes it like quick sort, where the base case is if the size is less than or equal to 16, it returns and exits. The partition algorithm is similar to the one described earlier, that being it gives an unsorted array, where elements on the left side are smaller than the pivot and the elements on the right are strictly larger than the pivot, in no particular order, the pivot in this situation being the middle element instead of last. After the partitions, it uses the "finalizeOrder" function to sort the array, essentially borrowing the insertion sorting algorithm. Insertion sort is implemented here with an outer for loop that creates a temporary movie array and a previous variable to track the current movie with the previous. Unlike bubble sort, comparing adjacent elements, comparing previous elements to the rest of the array. The combination of these two algorithms gives us the best and average case of O(n log n) and a worst case time complexity of O(n^2). Since we are using quick sort, the average time complexity is O(log n) and the worst space complexity is O(n).

**TopBoxOfficeRanker**

This algorithm borrows from the selection sorting algorithm with the best, worst, and average case being O(n^2). This algorithm searches for the minimum element and swaps that element into the correct position. Since there are no temporary arrays being created or being copied into, the space complexity is just O(1).

2. **Empirical Benchmarks: Include a table or plot showing data size (N) vs Execution time (ms) across your test runs using SortMetrics. Discuss whether your results match the theoretical Big-O curves**

Algorithms that used SortMetrics:

- Merge
- Quick
- sequenceStitcher
- Adaptive reel
- topBoxOfficeRanker

Firstly, I used python to plot these and made a data set containing the information from the executed Main.java using google sheets. I used histograms initially to plot the data, but given that the "n" inputs only contain two categories (10 vs 8000), using a line graph was a lot easier to see. However, there is one con of visualizing data using a line plot, it's that since we have two data points, the line graph draws a perfect line between the two points. But since the theoretical Big-O curves aren't linear, I would say that this result is really hard to read straight from the line graph below. Going off worse case:

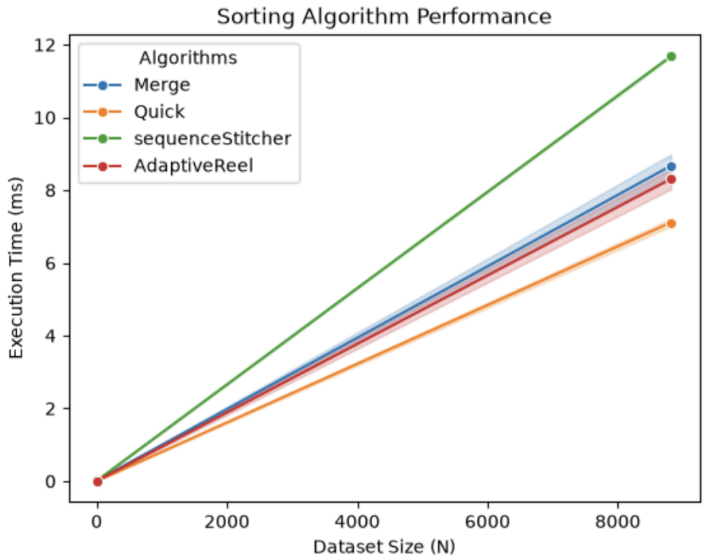
While on paper, Merge Sort has a time complexity of O(n log n) for all cases, Quick Sort has the same average time complexity. In my earlier test runs, Quick Sort was the fastest algorithm as n grew to approximately 8,814. However, this does not contradict the theoretical time complexities, since Big-O describes how an algorithm scales rather than its exact execution time. While these results don't "prove" the theoretical time complexity, they don't contradict it either. Given that I only tested two different values of n, the line graph is not enough to accurately identify the theoretical Big-O curves. However, the results are still consistent with the possibility that the algorithms follow their expected time complexities. The worst-case complexity does not necessarily occur for every dataset.

- Merge: O(n log n)
- Quick: O(n^2)
- SequenceStitcher: O(n log n)
- AdaptiveReelSort: O(n^2)
- TopBoxOfficeRanker: O(n^2)

[29]:
```
df_filtered = df[~df["Algorithms"].str.contains("TopBoxOffice", case=False, na=False)]
```

[30]:
```
sns.lineplot(data =df_filtered, x='n', y='Execution Time (ms)', hue = 'Algorithms', marker = 'o')

plt.title("Sorting Algorithm Performance")
plt.xlabel("Dataset Size (N)")
plt.ylabel("Execution Time (ms)")
plt.show()
```



[33]:
```
df.sort_values(by ='Execution Time (ms)' ).query('n > 10')
```

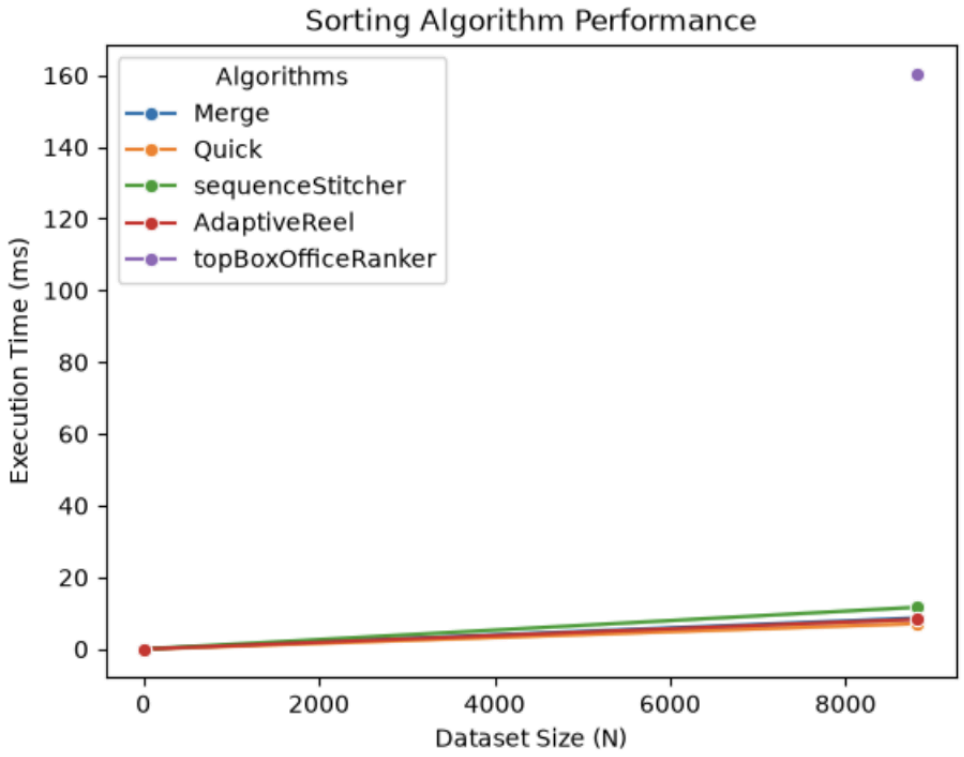
[33]:

| | Algorithms | n | Execution Time (ms) |
|---|---|---|---|
| **12** | Quick | 8814 | 7.009 |
| **3** | Quick | 8814 | 7.211 |
| **16** | AdaptiveReel | 8814 | 8.029 |
| **10** | Merge | 8814 | 8.355 |
| **7** | AdaptiveReel | 8814 | 8.581 |
| **1** | Merge | 8814 | 8.980 |
| **14** | sequenceStitcher | 8814 | 11.657 |
| **5** | sequenceStitcher | 8814 | 11.701 |
| **8** | topBoxOfficeRanker | 8814 | 160.509 |

[28]:
```
import matplotlib.pyplot as plt
import seaborn as sns

sns.lineplot(data =df, x='n', y='Execution Time (ms)', hue = 'Algorithms', marker = 'o')

plt.title("Sorting Algorithm Performance")
plt.xlabel("Dataset Size (N)")
plt.ylabel("Execution Time (ms)")
plt.show()
```




3. **Stress Test Results: Analyze how your sorting algorithms worked on edge cases including reverse-sorted and heavy duplicate values. Did any algorithm experience runtime spikes or stack overflow errors? If so**

**why? If your implementation avoided runtime spikes or stack overflows, explain how your pivot selection strategy protected against the worst-case big-O bounds.**

- Mainapp.java does not contain these calls for reverse ordered or heavily duplicated values. Unsure of what to do.

4. **Router Justification: Explain the decision rules of your SmartRouter. At what dataset size N or pre-sorted threshold does your router switch from an iterative/hybrid algorithm to a divide-and-conquer approach?**

For my smart router, it selects the algorithm based on the dataset size, with n=10. Essentially, if the size of the data set is less than or equal to 10, it uses “SequenceStitcher”, or bubble sort. And if n is greater than 10, it selects “AdaptiveReelSort” which is a combination of quick sort through partitioning and recursion with Insertion Sort, sorting post partition. Essentially, this hybrid makes it so that for a large data set, it partitions using divide/conquer, then uses insertion sort to select the minimum value from left to right to put into their respective place. Choosing n=10 was just a personal choice, no significant reason why. Using the small sample as a variable: “int smallSample = (int) (0.05 * movies.length);” this code says that it checks the first five percent of the array, then looping through the number of small samples, it compares the order. Using the “inversionCount” variable, if ten percent of the elements are out of order, then it chooses “AdaptiveReelSort”, and vice versa for “SequenceStitcher”.